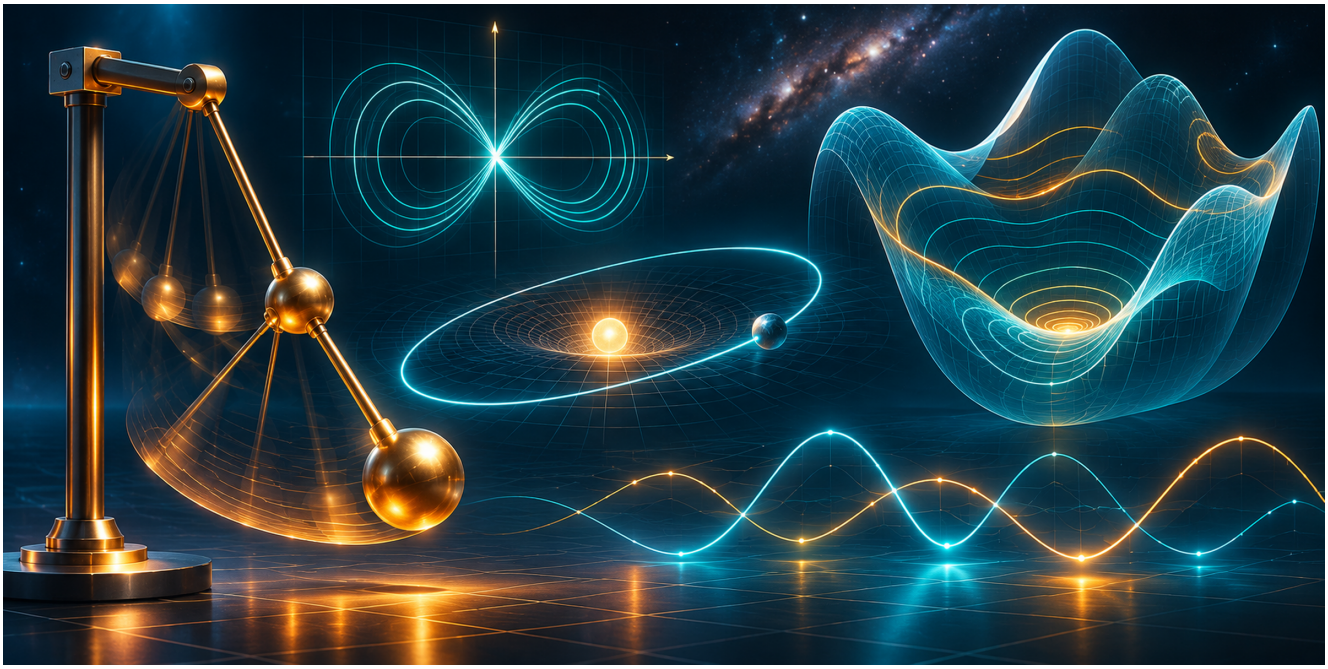
Computational Testing of Conservation-Law Candidates with paceval

An independently checked numerical demonstration using known dynamical laws, synthetic trajectories and original paceval Multi-Graph-Batch execution.



Conceptual scientific artwork. Pendulum, orbit and mathematical surfaces are design illustrations; the numbered result plots in this paper come from the executable numerical output.

EXECUTED SCOPE	ACTUAL VERIFIED RESULT
Candidate mathematical expressions	36 fixed candidates across 3 benchmark dynamical systems
Real paceval execution	144 persistent graphs; 645,168 Multi-Graph-Batch results
Independent audits	2,304 checks at 80 decimal digits; 288 candidate/noise/split evaluations
Numerical result	12 predeclared true invariants recognized; 24 negative controls rejected in controlled tests

Research boundary: conditional verification against exact known vector fields; no unknown physical law discovered, no measured industrial installation and no performance-speed claim.

1. The scientific and industrial use case

Manufacturing digital twins, vibration controllers, orbital systems and physics-guided machine learning depend on mathematical models that should respect basic dynamical constraints. A candidate invariant can be useful as a computational consistency check, but observing small variation on one trajectory alone does not establish a physical conservation law.

The user-supplied independent proposal isolates an intentionally simplified local balance expression: R = |dE/dt + div(flux) - source|. This paper selects the ordinary-differential-equation analogue: evaluate an explicit total derivative along the modeled vector field. The application-level task is to distinguish known invariants from plausible false candidates across independent simulated trajectories and noise levels.

Context and verified literature

Brunton, Proctor and Kutz (2016) introduced sparse identification of nonlinear dynamics (SINDy) as a broader equation-discovery framework [2]. Subsequent work on neural deflation studies the discovery of conservation laws directly from trajectories [3]. These references motivate the importance of independent, parsimonious candidate tests; this experiment does not claim novelty over those discovery methods.

Clear boundary between verification and discovery

QUESTION	ANSWER IN THIS PAPER
Known dynamical vector fields?	Yes: exact oscillator and Kepler equations are provided.
Predefined candidate functions?	Yes: 36 explicit analytic expressions, including negative controls.
Can paceval evaluate them?	Yes: each invariant, its unsimplified chain rule and two mathematical scores.
Does it infer a new unknown law?	No; unseen systems, unknown dynamics, numerical time derivatives and physical measurements are not evaluated.

2. Mathematical definition of conservation

Let x(t) be a state vector and dx/dt = f(x,t) a specified dynamical system. A differentiable candidate I(x,t) is an invariant of this defined model if its total derivative vanishes throughout a valid state-time domain, not only at one sampled trajectory.

$$\frac{dI}{dt} = \frac{\partial I}{\partial t} + \nabla I \cdot f(x,t) = 0$$

We explicitly register the unsimplified component-wise chain rule rather than substituting the algebraically simplified constant zero. Thus the original paceval engine computes numerical additions, products, powers, reciprocal/radial functions and exponentials in a genuine cancellation test.

Three analytically controlled dynamical systems

SYSTEM	VECTOR FIELD AND PHYSICAL INTERPRETATION
Anisotropic 2D oscillator	dx/dt = v; dv/dt = -x; dy/dt = w; dw/dt = -4y. Mode frequencies 1 and 2.
Planar Kepler problem	dx/dt = v; dy/dt = w; dv/dt = -mu*x/r^3; dw/dt = -mu*y/r^3; r = sqrt(x^2+y^2).
Underdamped oscillator	dx/dt = v; dv/dt = -x - 2*zeta*v, with zeta = 0.075 and an explicit time-dependent invariant.

The initial states and smooth trajectories are generated analytically by C++ to avoid accidentally conflating numerical integration drift with conservation-law violations. Measured state vectors are separately perturbed for the declared observational-noise experiments.

3. Candidate library and physical interpretation

A. Anisotropic oscillator

$$E_x = \frac{x^2 + v^2}{2}, \quad E_y = \frac{4y^2 + w^2}{2}$$

The two mode energies are individually conserved, as are their sum and any differentiable function thereof. We include twelve oscillator candidates: four true invariant functions, including an algebraically dependent square, and eight counterexamples such as x, v, x*v and v^2.

B. Planar Kepler dynamics

$$H = \frac{v^2 + w^2}{2} - \frac{\mu}{r}, \qquad L = xw - yv$$

Besides specific energy H and angular momentum L, both in-plane components of the Laplace-Runge-Lenz (LRL) vector are tested. The derived H^2 and L^2 candidates are mathematically conserved but not additional independent conservation laws. Six predeclared invariants are accompanied by six negative controls.

C. Damped oscillator

$$I(t) = e^{2\zeta t}\left(v^2 + 2\zeta xv + x^2\right)$$

Raw mechanical energy dissipates and is therefore a negative control. The explicitly time-dependent I(t), and its square, remain conserved under the *declared* damped equation. This illustrates why a valid conserved quantity may require explicit time dependence and why arbitrary stationary-energy claims are incorrect.

There are 12 positive candidate function labels in total (4+6+2); that is not a count of functionally independent invariants. The candidate set was fixed before running the numerical evaluation.

4. Actual original paceval integration

The executable includes the original paceval_main.h and paceval_main.cpp interfaces, and dynamically accesses the separately supplied compatible original Linux library. All 36 candidate functions are registered with long-lived handles using paceval_CreateComputation, without creation inside the trajectory loop.

GRAPH FOR EACH CANDIDATE	EXECUTION OWNER
1. Candidate value I(x,t)	Real persistent paceval computation
2. Explicit Lie derivative dI/dt	Real persistent paceval graph: full unsimplified chain rule
3. Normalized |dI/dt| / (1 + |I|)	Real persistent paceval graph; absolute values built from sqrt(square)
4. Squared Lie derivative (dI/dt)^2	Real persistent paceval graph
Time sequence, reference trajectory, noise	C++ host: analytic solution and deterministic input generation
RMS across many observations; selection	C++ host: aggregation and fixed thresholds, not paceval formulas

The 144 registered graphs are evaluated by the true paceval_dGetMultipleComputationsResultsExt API. Every individual SDK error code is checked and the tested library’s non-standard batch bool return convention is respected. Each model group uses 48 graph handles on blocks of trajectory or surface points.

Versioned formula declarations and exact ownership

The complete source strings are committed to results/formula_registry.csv and src/generated_registry.h, with their source generator src/generate_registry.py. The machine-readable math_execution_inventory.csv identifies all execution owners and distinct API paths. The full source and its graph inventory are part of the project archive.

5. Reproducible numerical experiment design

CONTROL	DECLARED VALUE
Benchmark systems / candidates	3 dynamical systems; 36 frozen candidate expressions
Noise levels	Standard deviations: 0, 0.005, 0.02, 0.05 in model-state units
Initial conditions	4 distinct analytic trajectories per system
Sample times	192 evenly spaced instants over normalized 0 to 14
Train/held-out separation	Trajectories 0–1 for training; 2–3 for holdout; fixed before tests
Candidate verification	Normalized Lie RMS < 1e-7 and mean per-trajectory drift < 0.002 + 12*sigma
3D demonstration	65 × 65 phase-space points with true paceval Lie evaluations

The thresholds are explicit demonstration-policy parameters, not rigorous floating-point enclosures, measured sensor-error models or evidence-optimized tuning. Gaussian state perturbations are deterministic for repeatability. Importantly, the noise does not alter the supplied exact ODE: the Lie expression evaluates the model at perturbed observed states. This is *not* a test of reconstructing a vector field from noisy data.

Independent controls

The program checks each per-result error, finite input domain, squared-derivative identity, and 6,440 invariant-Lie reference values. A separate 80-digit mpmath program parses the exported literal expressions independently and checks 2,304 actual paceval result entries, including held-out phase-space samples.

6. Executed outcomes and scientific meaning

GROUNDED OUTPUT	ACTUAL OBSERVED RESULT
Persistent original paceval graphs	144 across 36 candidate laws
Actual Multi-Graph-Batch results	645,168
Batch API invocations	161
Scalar SDK spot checks	36
Distinct vector inputs, including surface	13441
Analytic invariant Lie checks	6440
Largest absolute known-Lie cancellation	1.78e-15
80-decimal-digit independent checks	2304
Train/held-out candidate/noise/split rows	288

Conditional recovery on a deliberately constructed candidate library

Across the locked benchmark, all known invariant formulas were recovered and all counterexamples rejected at each declared noise level for both data splits. The held-out precision and recall are 1.00 *for this exact, intentionally separable and previously labeled mathematical library*. The direct Lie-derivative score uses the complete known vector field, so this result should never be extrapolated to unknown-physics discovery accuracy or real noisy derivative estimates.

Numerical result rather than performance marketing

This paper reports mathematical executability, reproducibility and domain-specific interpretation, not measured speedups or C++-versus-paceval numerical-difference tables. The independent checks are shipped for technical audit and do not constitute experimental physical measurements.

7. Computed results I: oscillator and orbit

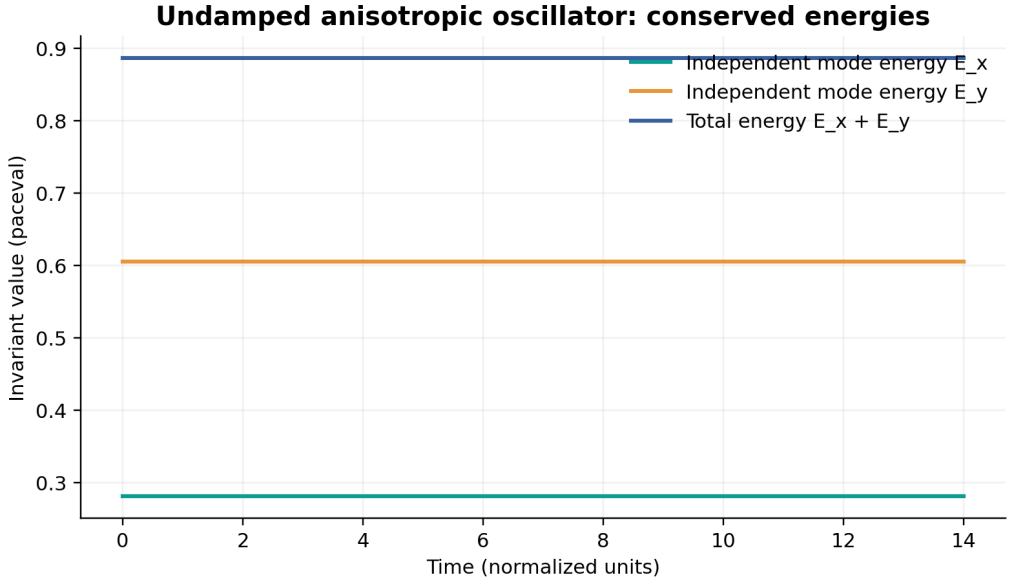


Figure 1. The two separately conserved oscillator mode energies and their sum, evaluated as independent original paceval expressions along one analytic trajectory.

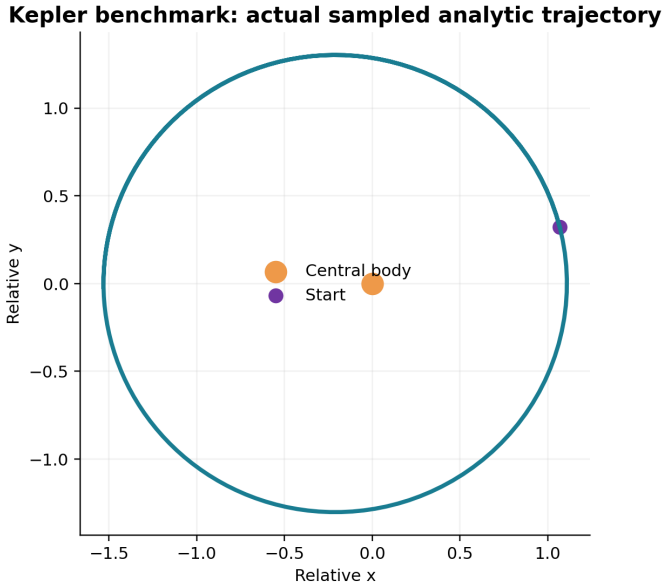


Figure 2. The exact source trajectory supplied to paceval for the Kepler model, shown in physical phase-space coordinates. It is synthetic benchmark input, not observed astronomical tracking.

8. Computed results II: two different kinds of conservation

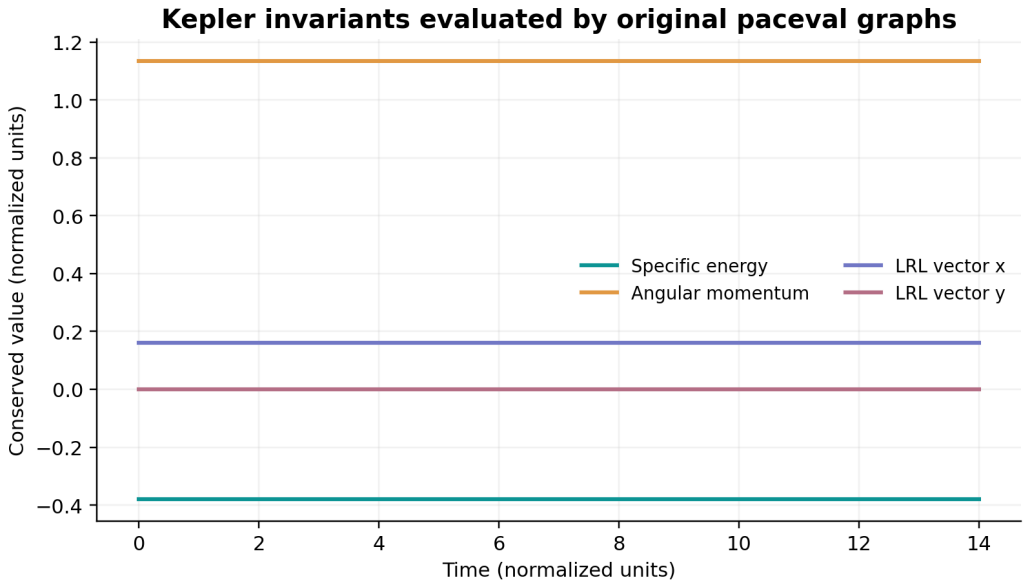


Figure 3. Actually evaluated Kepler energy, angular momentum and two Laplace-Runge-Lenz components. The drawn quantities have different units and are displayed together only for numerical constancy.

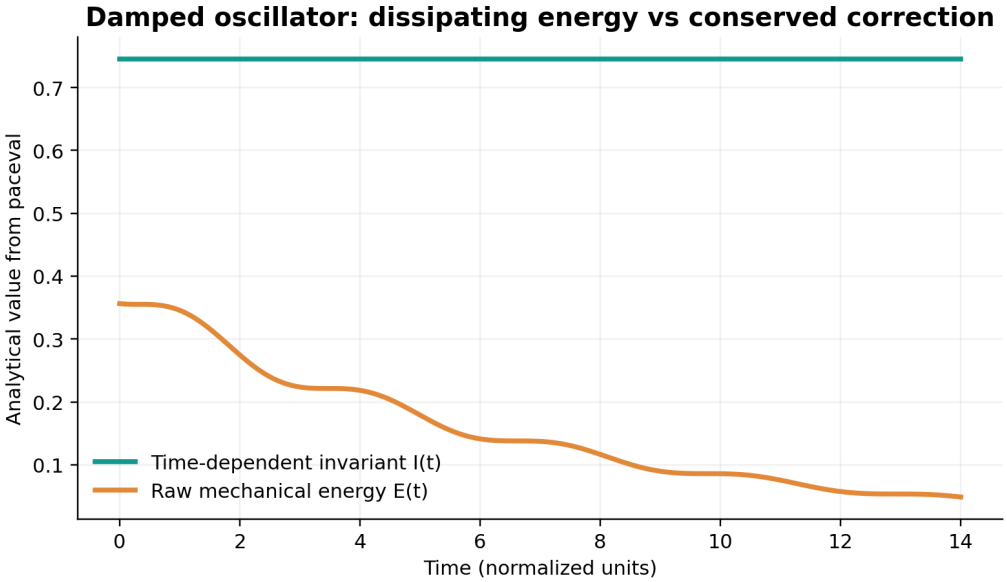


Figure 4. The raw mechanical energy is decreasing while a mathematically correct time-weighted invariant stays constant under the specified damped oscillator.

9. Computed results III: 3D violation surface

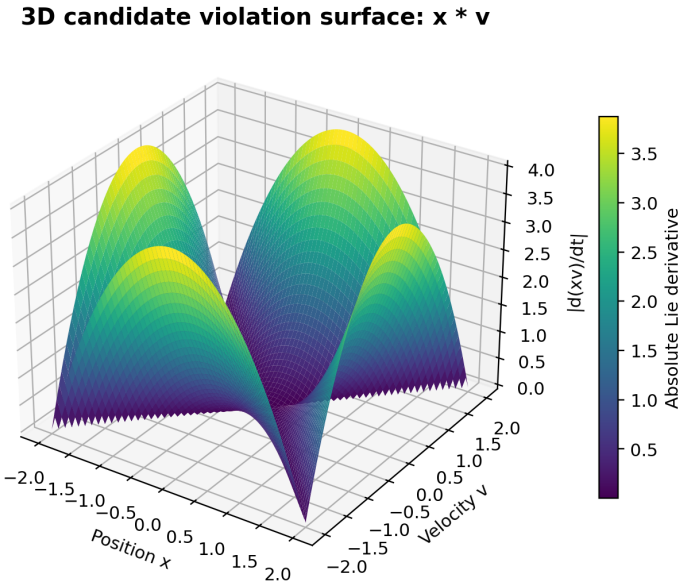


Figure 5. A 65 by 65 real paceval batch evaluation of the unsimplified Lie derivative for the FALSE candidate x*v in an undamped oscillator. The surface is not a physical potential.

A genuine invariant has a vanishing derivative throughout its defined model domain. Conversely, x*v has derivative v^2 - x^2 under x-dot=v and v-dot=-x; its zero-valued curves alone do not make it globally conserved. This is why analyzing one trajectory, or a small sampled residual, is insufficient.

10. Computed results IV: contours and candidate separation

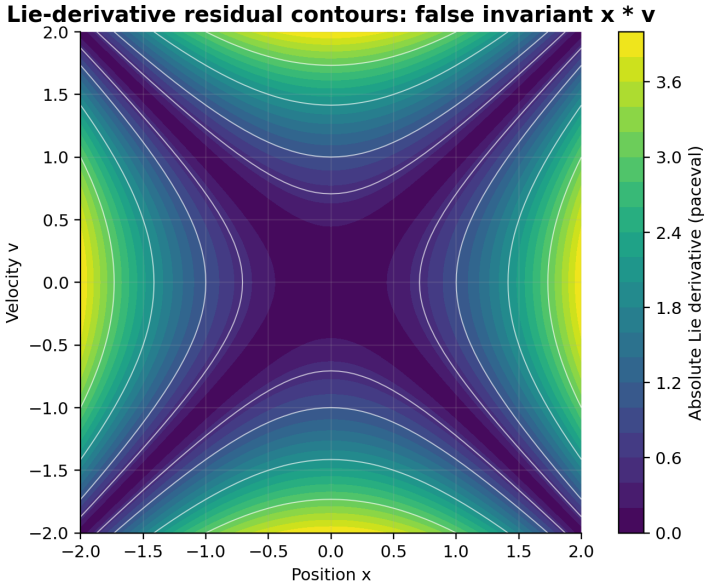


Figure 6. True paceval contour map of absolute Lie derivative for the oscillator negative-control expression x*v. Local zeros and globally nonzero regions coexist.

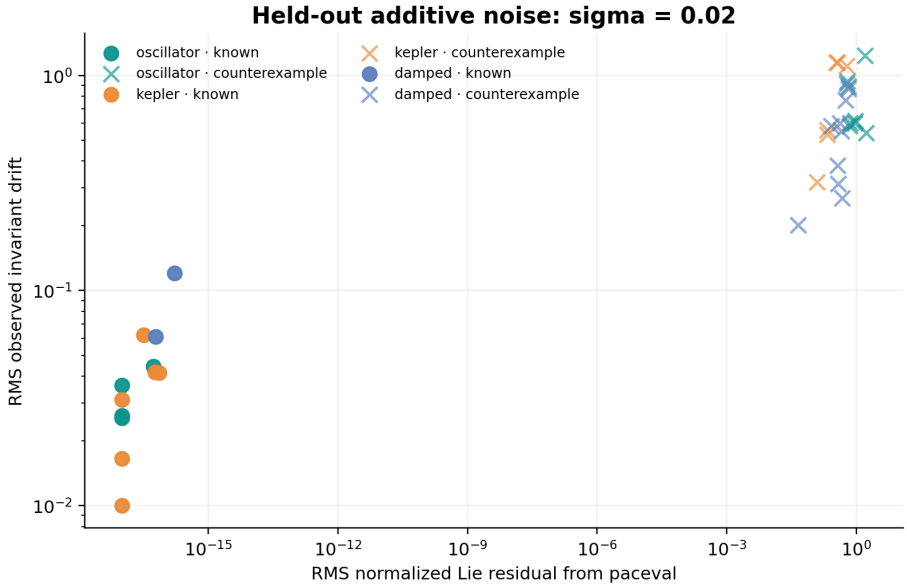


Figure 7. Held-out candidate scores at additive state-noise sigma = 0.02: recognized mathematical invariants versus the fixed negative controls, using actual paceval Lie and invariant values.

11. Computed results V: observational noise

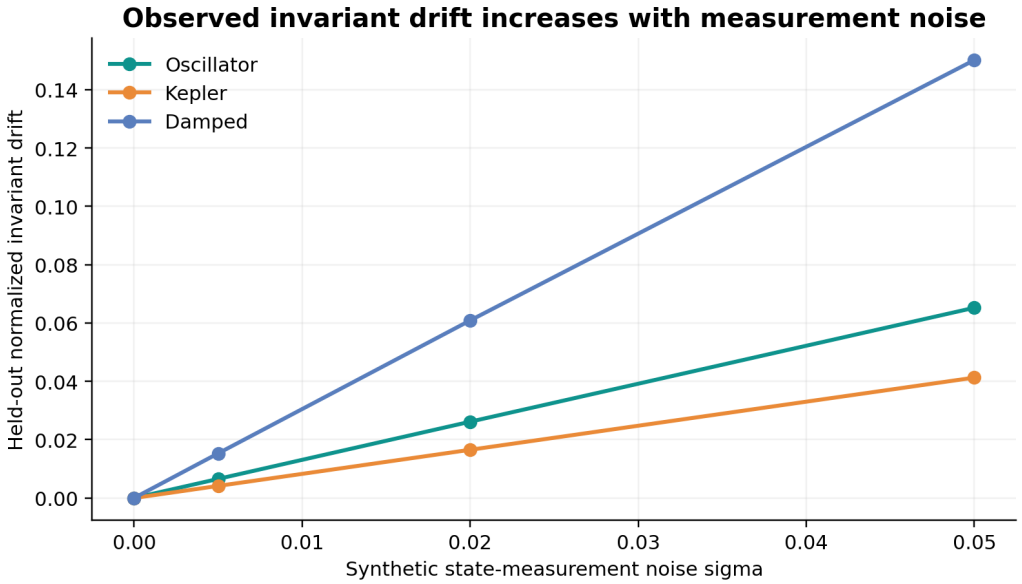


Figure 8. Observed trajectory drift rises with injected state-measurement noise, even though the theoretical analytic invariants of the exact ODE remain valid.

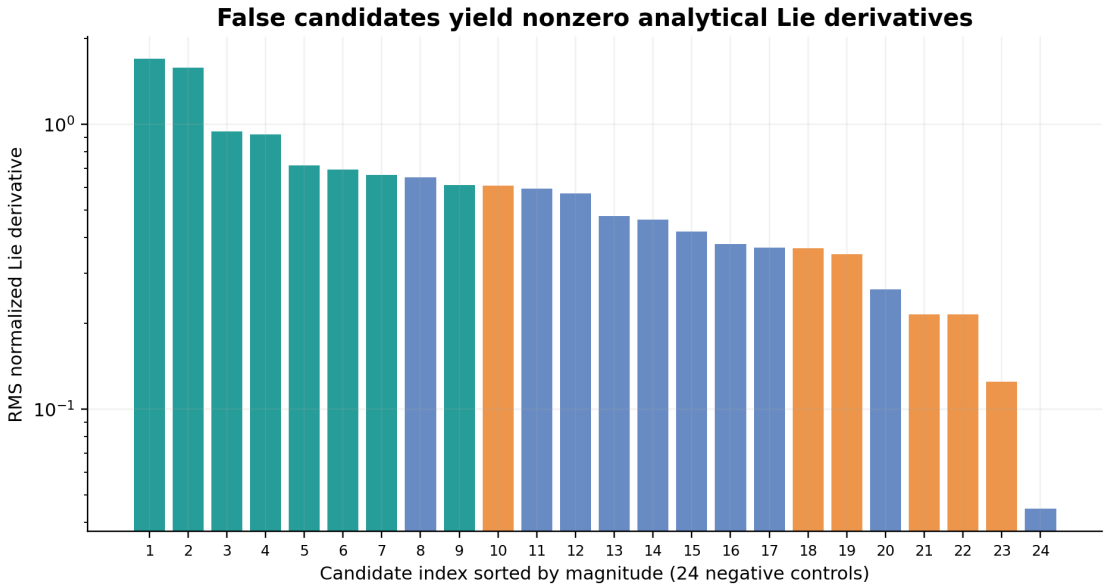


Figure 9. The 24 known non-invariants have strictly nonzero normalized Lie derivative RMS scores in the held-out data under the declared benchmark.

12. Computed results VI: conditional recall and graph inventory

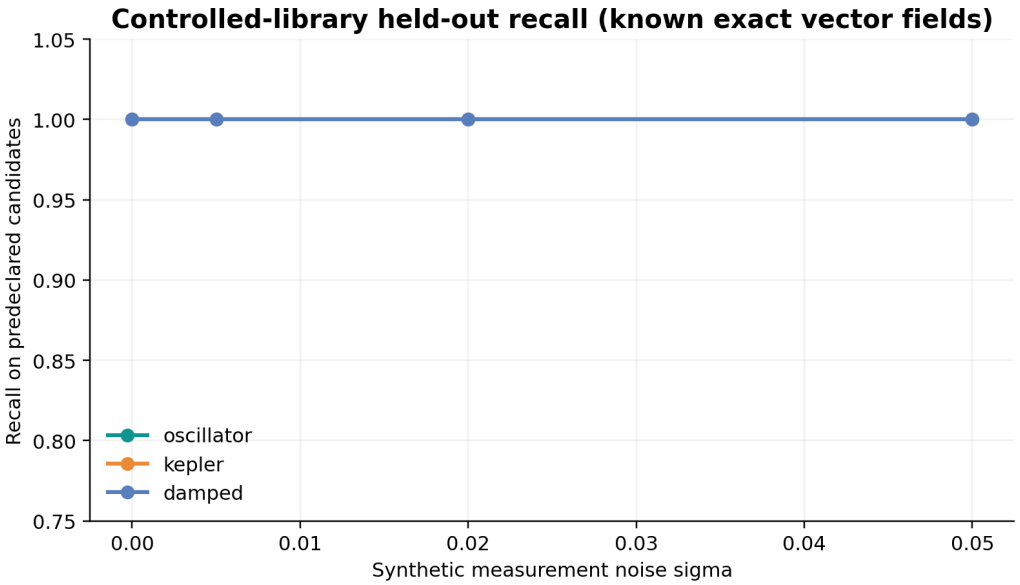


Figure 10. Perfect held-out recall in this known-model/closed-candidate benchmark. This is NOT an estimate of out-of-family discovery performance.

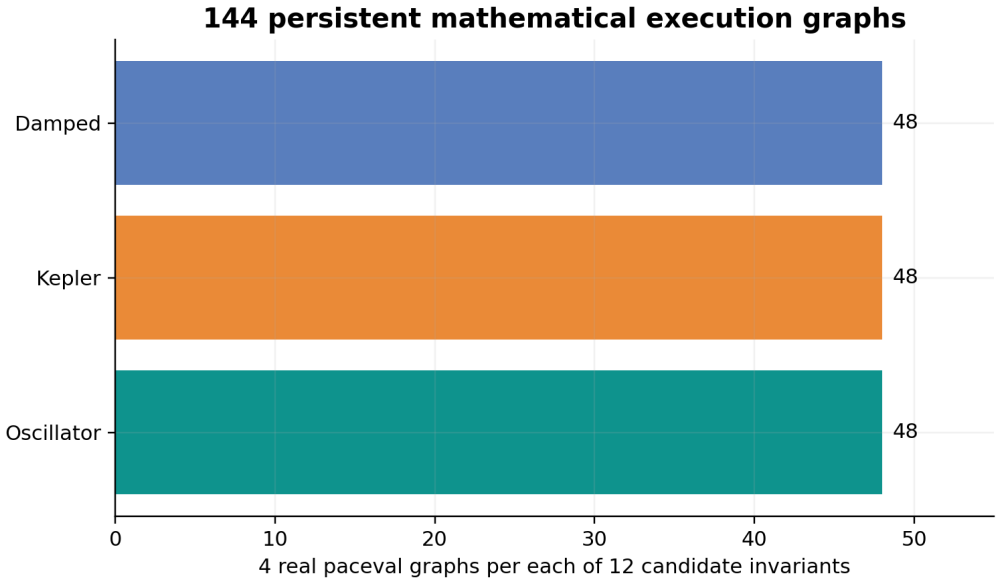


Figure 11. All 144 persistent real paceval graphs are accounted for, including candidate values, explicit total derivatives and two numerical residual scores.

13. Verification, methodological safeguards and limitations

AUDIT	RESULT
Original C++ program + SDK	Built with original paceval_main.cpp and external original library
CMake/CTest	Passed with genuine original library
Analytic Lie checks	6,440 passed, max |dI/dt| = 1.78e-15
Independent 80-digit mpmath	2,304 results passed
Precision/recall and control labels	36 fixed candidates, four noise levels, two splits, 288 rows
Reproducibility	Repeat byte-identical CSVs verified separately; hashes stored in package

Interpretation and what this does not establish

With an exactly known vector field and analytically supplied chain-rule derivatives, the test is a candidate-verification benchmark. It is not autonomous discovery, a successful inference of true dynamics from noisy trajectories, a rigorous proof of physical truth from finite data, or a measurement of laboratory energy conservation. Even a zero-valued Lie derivative for a formally defined ODE only certifies compatibility with that ODE and its domain assumptions.

Some recovered candidates are dependent functions of others. An independent-function discovery algorithm would require rank tests or another equivalence reduction, not simply counting 12 positive labels. Long noisy trajectories, solver discretization, model misspecification and sensor calibration remain future experiments.

14. Reproduction, research roadmap and references

Reproducible project

Build with CMake from src/CMakeLists.txt after supplying a separately obtained original paceval Linux library. Typical commands: cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so; cmake --build build -j; ctest --test-dir build --output-on-failure; ./build/conservation_law_candidates --out .

Independent reproduction runs src/high_precision_reference.py and src/verify_results.py, regenerates numbered scientific images using src/make_figures.py, and rebuilds this report with src/make_whitepaper.py. The checked-in original paceval SDK interfaces preserve their existing notices; no external runtime binary is redistributed.

Scientifically useful next experiments

A realistic next stage would fit or discover f from incomplete/noisy observations, compare strong/weak-form differentiation, rank independent invariant families, extend to nonintegrable and forced or stochastic systems, quantify false-discovery rates on held-out physical datasets, and benchmark against modern constrained symbolic-regression or neural-law approaches. These tasks are not claimed here.

Research references

[1] WP_041 Computational Testing of Conservation-Law Candidates, user-supplied independent proposal, November 2026.

[2] Brunton, Proctor, Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems. PNAS 113 (2016). DOI: 10.1073/pnas.1517384113.

[3] Data-driven discovery of conservation laws from trajectories via neural deflation. Communications in Nonlinear Science and Numerical Simulation 143 (2025), 108563. DOI: 10.1016/j.cnsns.2024.108563.

[4] paceval mathematical engine, original SDK API and Multi-Graph-Batch interface: https://paceval.com/api/

[5] paceval open research repository: https://github.com/paceval/paceval

Conclusion

Verified outcome: original paceval computes 36 reusable candidate laws, their unsimplified analytic Lie derivatives, and numerical validity scores via 144 actual persistent graphs across controlled physical examples and held-out synthetic trajectories. **Scientific boundary:** no unknown law, external sensor process or industrial advantage is thereby discovered.